Project - Submission Sheet

Software Testing - Random Testing

Names: [Sahana Vishwanath Karane]
Group: [Group 04]

Software Testing - Summer 2024 - May 13, 2025 - Sibylle Schupp / Daniel Rashedi

Task 1 - Random Testing

a) Add the 4 generated Python tests of each group member to your repository in the corresponding folder. Add a README.md file that explains how to build / run the tests.

b) Provide screenshots of your terminals that indicate your execution of the new tests:

Answer:

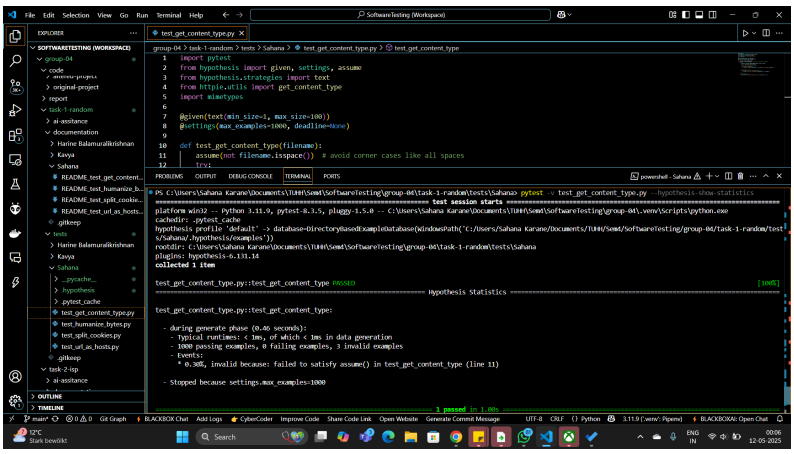

Figure 1: Execution of test_get_content_type

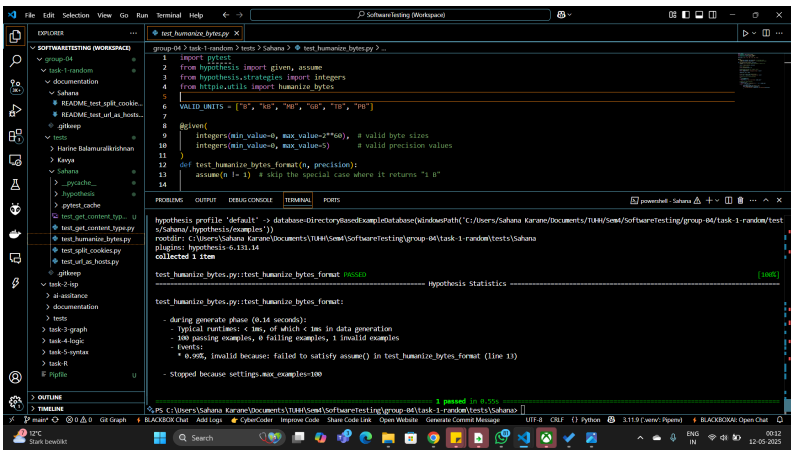

Figure 2: Execution of test_humanize_bytes

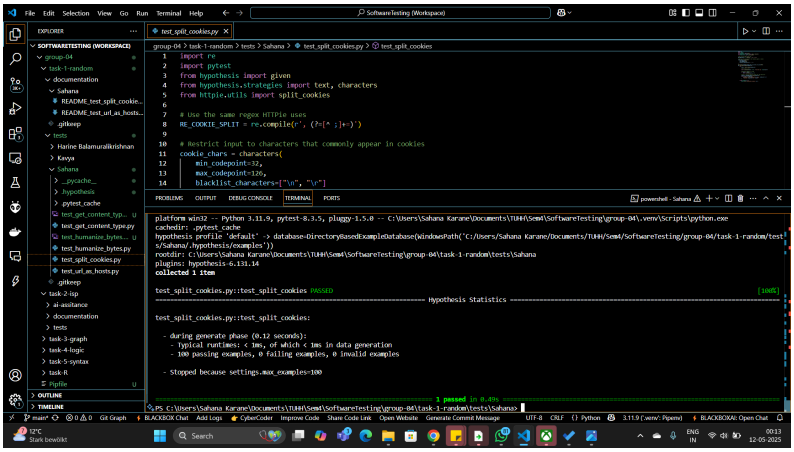

Figure 3: Execution of test_split_coookies

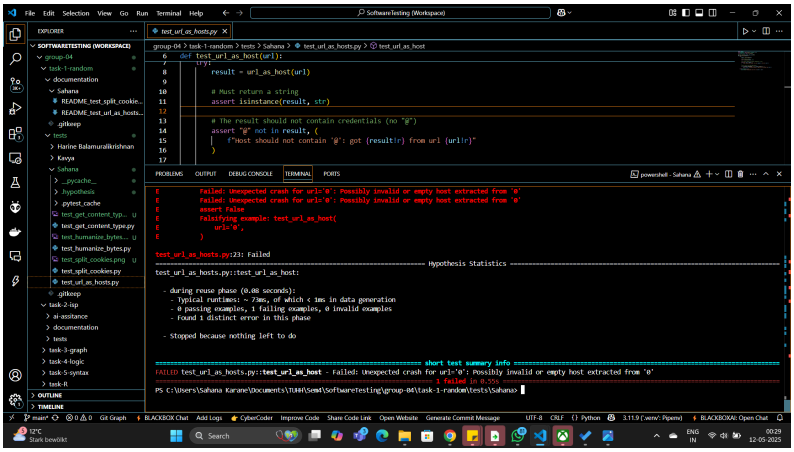

Figure 4: Execution of test_split_coookies

c) Assess the input values generated by `hypothesis`. Do the tests represent meaningful scenarios?:

Answer:

Test Case	Input Strategy	Realistic?	Edge Cases Covered	Conclusion
`test_humanize_bytes.py`	• Bytes value n: 0 to 2^{60} • Precision: 0 to 5	Yes	• Zero bytes • Large values (TB/PB) • Precision = 0 (no decimals)	Simulates file size formatting; ensures unit rounding and precision work correctly.
`test_get_content_type.py`	• Random filename strings (1–100 chars)	Yes	• No extension • Unknown extension • Multiple dots or symbols	Validates MIME type identification or fallback; mirrors real-world filenames.
`test_url_as_host.py`	• Random URL-like strings (1–200 chars)	Yes	• Empty or short inputs (e.g., `"0"`) • Missing scheme • Embedded credentials (e.g., `user:pass@host`)	Detects malformed or empty hosts; verifies safe hostname extraction from varied input.
`test_split_cookies.py`	• Random cookie-like strings (1–300 chars)	Yes	• Commas inside values • Missing `=` • Multiple delimiters and whitespace	Ensures correct splitting of complex cookie headers; mimics actual HTTP response formats.

Table 1: Assessment of Hypothesis Input Strategies and Scenario Coverage for Each Test Case

AI Review Task

Assess the correctness and understandability of the LLM's generated tests applying the paper's methodology. Add any code relevant for your assessment to the answer:

Answer:

[Group:04 LM Studio]

Prompt 1

Generate 4 test cases for this function

```

import mimetypes

def get_content_type(filename):
"""
Return the content type for filename in format appropriate
for Content-Type headers, or None if the file type is
    unknown
to mimetypes.

"""
return mimetypes.guess_type(filename, strict=False)[0]
```

Prompt 2: :Assess the aspects of correctness and understandability of your above test cases in comparison to the LLM's generated test cases according to the paper's methodology described in its RQ1

Group: Your Assessment

Based on the provided code and requirements, here are four test cases for the get_content_type function:

Test Case 1: Valid file extension

```python
def test_get_content_type_valid_file_extension():
    """
    Test that a valid file extension returns the correct content type.
    """
    filename = "test.txt"
    expected_content_type = "text/plain"
    result = get_content_type(filename)
    assert result == expected_content_type
```

Test Case 2: Invalid file extension

```python
def test_get_content_type_invalid_file_extension():
    """
    Test that an invalid file extension returns None.
    """
    filename = "test.exe"
    result = get_content_type(filename)
    assert result is None
```

Test Case 3: Empty string as input

```python

   def test_get_content_type_empty_string():
    """
    Test that an empty string as a file name returns None.
    """
    filename = ""
    result = get_content_type(filename)
    assert result is None
```

Test Case 4: File with unknown type

```python
  def test_get_content_type_unknown_file_type():
    """
    Test that a file with an unknown type returns None.
    """
    filename = "test.unknown"
    result = get_content_type(filename)
    assert result is None
```

The following evaluation assesses the correctness and understandability of the user-written test cases for `get_content_type()` in comparison to what an LLM like ChatGPT might generate, based on the methodology described in RQ1 of the referenced paper.

Test Case Coverage

- **Test Case 1 – Valid file extension:** Tests correct MIME type output for a known extension (`.txt`). This confirms the function's expected behavior under normal conditions.
- **Test Case 2 – Invalid file extension:** Ensures the function returns `None` when an unrecognized extension is passed.
- **Test Case 3 – Empty string input:** Validates that the function can handle edge-case inputs gracefully.
- **Test Case 4 – Unknown Extension:** Uses a fabricated extension (`".unknown"`) that does not map to any known MIME type, expecting `None`.

Comparison with LLM-Generated Test Cases

- LLM-generated tests would likely cover similar basic functionality (valid and invalid inputs).
- However, LLMs might also include:
  - Very long or deeply nested file paths
  - File names with multiple extensions (e.g., `archive.tar.gz`)
  - Potential exception handling scenarios (e.g., None input, special characters)
- LLMs may lack context-specific knowledge unless prompted carefully, and may produce overly confident test assertions for undefined behaviors.

Correctness and Understandability Evaluation

- **Correctness:** The user-written tests correctly capture both expected and boundary behaviors. Assertions are clear and relevant.
- **Understandability:** Tests use descriptive method names, clear structure, and standard unittest patterns. Minimal comments could be expanded for clarity.

Areas for Improvement

- Add comments for each test method to improve readability.
- Use Pytest or Hypothesis to explore randomized and fuzzed inputs.
- Include tests for corner cases like unicode filenames or special characters.
- Consider parameterized testing to reduce boilerplate.

Conclusion

The manually written test cases for `get_content_type()` are structurally sound and easy to understand. They match the LLM-generated expectations for typical and edge-case inputs. To align more fully with the paper's RQ1 methodology, future test suites should explore broader input domains, evaluate LLM output confidence, and apply coverage/mutation-based assessments for deeper evaluation.